

Mipmap

In computer graphics, a **mipmap** (*mip* being an acronym of the Latin phrase *multum in parvo*, meaning "much in little"^[1]) is a pre-calculated, optimized sequence of images, each of which has an image resolution which is a factor of two smaller than the previous.^{[2][3][4]} Their use is known as *mipmapping*.

They are intended to increase rendering speed and reduce aliasing artifacts. A high-resolution mipmap image is used for high-density samples, such as for objects close to the camera; lower-resolution images are used as the object appears farther away. This is a more efficient way of downscaling a texture than sampling all texels in the original texture that would contribute to a screen pixel; it is faster to take a constant number of samples from the appropriately downfiltered textures. Since mipmaps, by definition, are pre-allocated, additional storage space is required to take advantage of them. They are also related to wavelet compression.

Mipmaps are widely used in 3D computer games, flight simulators, other 3D imaging systems for texture filtering, and 2D and 3D GIS software. Mipmap textures are used in 3D scenes to decrease the time required to render a scene. They also improve image quality by reducing aliasing and Moiré patterns that occur at large viewing distances,^[5] at the cost of 33% more memory per texture.

History

Mipmapping was invented by Lance Williams in 1983 and is described in his paper *Pyramidal parametrics*.^[1] From the abstract: "This paper advances a 'pyramidal parametric' prefiltering and sampling geometry which minimizes aliasing effects and assures continuity within and between target images." The referenced pyramid can be imagined as the set of mipmaps stacked in front of each other.

The first patent issued on Mipmap and texture generation was in 1983 by Johnson Yan, Nicholas Szabo, and Lish-Yann Chen of Link Flight Simulation (Singer). Using their approach, texture could be generated and superimposed on surfaces (curvilinear and planar) of any orientation and could be done in real-time. Texture patterns could be modeled suggestive of the real world material they were intended to represent in a continuous way and free of aliasing, ultimately providing level of detail and gradual (imperceptible) detail level transitions. Texture generating became repeatable and coherent from frame to frame and remained in correct perspective and appropriate occultation. Because the application of real time texturing was applied to early three dimensional flight simulator CGI systems, and texture being a prerequisite for realistic graphics, this patent became widely cited and many of these techniques were later applied in graphics computing and gaming as applications expanded over the years.^[6]

The origin of the term mipmap is an initialism of the Latin phrase *multum in parvo* ("much in little"), and map, modeled on bitmap.^[1] The term *pyramids* is still commonly used in a GIS context. In GIS software, pyramids are primarily used for speeding up rendering times.

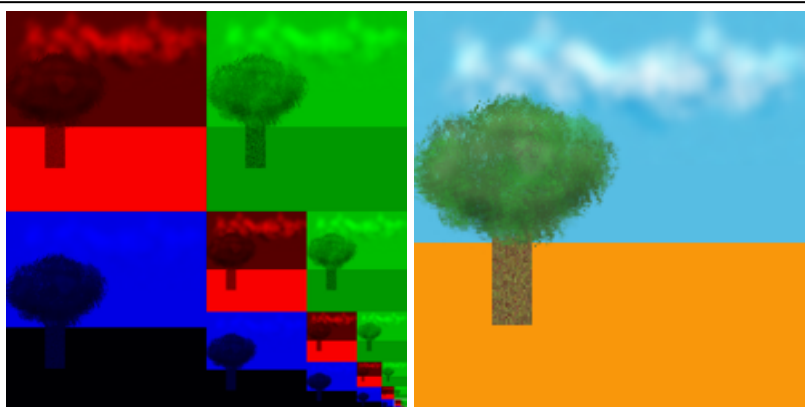
Mechanism

Each bitmap image of the mipmap set is a downsized duplicate of the main texture, but at a certain reduced level of detail. Although the main texture would still be used when the view is sufficient to render it in full detail, the renderer will switch to a suitable mipmap image (or in fact, interpolate between the two nearest, if trilinear filtering is activated) when the texture is viewed from a distance or at a small size. Rendering speed increases since the number of texture pixels (texels) being processed per display pixel can be much lower for similar results with the simpler mipmap textures. If using a limited number of texture samples per display pixel (as is the case with bilinear filtering) then artifacts are reduced since the mipmap images are effectively already anti-aliased. Scaling down and up is made more efficient with mipmaps as well.



Example mipmap image storage: the principal image on the left is accompanied by filtered copies of reduced size.

If the texture has a basic size of 256 by 256 pixels, then the associated mipmap set may contain a series of 8 images, each one-fourth the total area of the previous one: 128×128 pixels, 64×64, 32×32, 16×16, 8×8, 4×4, 2×2, 1×1 (a single pixel). If, for example, a scene is rendering this texture in a space of 40×40 pixels, then either a scaled-up version of the 32×32 (without trilinear interpolation) or an interpolation of the 64×64 and the 32×32 mipmaps (with trilinear interpolation) would be used. The simplest way to generate these textures is by successive averaging; however, more sophisticated algorithms (perhaps based on signal processing and Fourier transforms) can also be used.



Showing each color channel of each level of an RGB mipmap as a separate plane (left) demonstrates that the whole mipmap forms a square of 4 times the area. As each plane requires $\frac{1}{3}$ the storage, mipmaps therefore require $\frac{4}{3}$ the memory; i.e., $\frac{1}{3} \approx 33\%$ more.

The increase in storage space required for all of these mipmaps is a third of the original texture, because the sum of the areas $\frac{1}{4} + \frac{1}{16} + \frac{1}{64} + \frac{1}{256} + \dots$ converges to $\frac{1}{3}$. In the case of an RGB image with three channels stored as separate planes, the total mipmap can be visualized as fitting neatly into a square area twice as large as the dimensions of the original image on each side (twice as large on each side is four times the original area - one plane of the original size for each of red, green and blue makes three

times the original area, and then since the smaller textures take 1/3 of the original, 1/3 of three is one, so they will take the same total space as just one of the original red, green, or blue planes). This is the inspiration for the tag *multum in parvo*.

Uses

Mipmaps are used for:

- Level of detail (LOD)^{[7][8]}
- Improving image quality. Rendering from large textures where only small, discontinuous subsets of texels are used can easily produce Moiré patterns;
- Speeding up rendering times, either by reducing the number of texels sampled to render each pixel, or increasing the memory locality of the samples taken;
- Reducing stress on the GPU or CPU.
- Water surface reflections^[9]

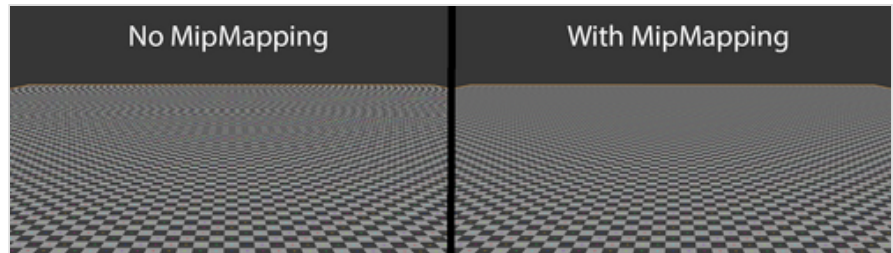


Image showing how mipmaps reduce aliasing at large distances (Aliasing causes a Moiré pattern in the left image.)

Anisotropic filtering

When a texture is viewed at a steep angle, the filtering should not be uniform in each direction (it should be anisotropic rather than isotropic), and a compromise resolution is required. If a higher resolution is used, the cache coherence goes down, and the aliasing is increased in one direction, but the image tends to be clearer. If a lower resolution is used, the cache coherence is improved, but the image is overly blurry. This would be a tradeoff of MIP level of detail (LOD) for aliasing vs blurriness. However anisotropic filtering attempts to resolve this trade-off by sampling a non isotropic texture footprint for each pixel rather than merely adjusting the MIP LOD. This non isotropic texture sampling requires either a more sophisticated storage scheme or a summation of more texture fetches at higher frequencies.^[10]

See also

- Anisotropic filtering – Method of enhancing the image quality of textures on surfaces of computer graphics
- Hierarchical modulation – Signal processing technique
- Pyramid (image processing) – Type of multi-scale signal representation
- Scale space – Framework for multi-scale signal representation
- Spatial anti-aliasing – Technique for reducing low-resolution image distortion
- Summed-area table – Type of data structure algorithm

References

1. Williams, Lance. "Pyramidal Parametrics" (<https://web.archive.org/web/20140414134825/http://staff.cs.psu.ac.th/iew/cs344-481/p1-williams.pdf>) (PDF). Archived from the original (<http://staff.cs.psu.ac.th/iew/cs344-481/p1-williams.pdf>) (PDF) on 2014-04-14. Retrieved 2012-09-25.
2. "Texture Filtering with Mipmaps (Direct3D 9)" (<https://docs.microsoft.com/en-us/windows/win32/direct3d9/texture-filtering-with-mipmaps>). *microsoft.com*. Microsoft.
3. "Texture Filtering with Mipmaps" ([https://docs.microsoft.com/en-us/previous-versions/aa921432\(v=msdn.10\)](https://docs.microsoft.com/en-us/previous-versions/aa921432(v=msdn.10))). *microsoft.com*. Microsoft. April 8, 2010.
4. "Mipmap Texturing" (https://cgl.ethz.ch/teaching/former/vc_master_06/Downloads/Mipmaps_1.pdf) (PDF). Retrieved December 10, 2019.
5. "Anti-Aliasing Problem and Mipmapping" (<https://textureingraphics.wordpress.com/what-is-texture-mapping/anti-aliasing-problem-and-mipmapping/>). *textureingraphics*. 2011-12-13. Retrieved 2019-02-21.
6. Johnson K. Yan, Nicholas S. Szabo, Lish-Yann Chen. "Texture Generation" (<https://patentimages.storage.googleapis.com/9d/3b/9f/603a55c4b7f692/US4615013.pdf>) (PDF).
7. "Multiple levels of Detail" (<https://people.cs.clemson.edu/~dhouse/courses/405/notes/OpenGL-mipmaps.pdf>) (PDF).
8. "D3D11_SAMPLER_DESC structure" ([https://msdn.microsoft.com/en-us/library/windows/desktop/ff476207\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ff476207(v=vs.85).aspx)). *microsoft.com*. Microsoft.
9. "Deconstructing the water effect in Super Mario Sunshine" (<https://blog.mecheye.net/2018/03/deconstructing-the-water-effect-in-super-mario-sunshine>). 3 March 2018. Retrieved 25 February 2023.
10. Olano, Marc; Mukherjee, Shrijeet]]; Dorbie, Angus. "Vertex-based Anisotropic Texturing" (<https://www.csee.umbc.edu/~olano/papers/vbat/vbat.pdf>) (PDF).

Retrieved from "<https://en.wikipedia.org/w/index.php?title=Mipmap&oldid=1334827336>"